

Contents

Movie Recommendation System:	2
Overview	2
Objective	2
Business Problem	2
Dataset Description:	2
Data Source:	3
Exploratory Data Analysis:	3
Building Recommendation system	6
Item-Item Recommendations using k-Nearest Neighbors (k-NN)	6
Movie Recommendation using SVD (Singular Value Decomposition)	7
Movie Recommendation Using Collaborative filtering	8
Conclusion:	9

Movie Recommendation System:

Overview

With an ever-expanding catalog of movies on digital platforms, users face the challenge of selecting content that matches their tastes. Recommendation systems play a critical role in enhancing user experience by suggesting relevant movies based on individual preferences. This project explores the development of a hybrid movie recommendation engine using the MovieLens 100k dataset, integrating collaborative filtering and content-based methods.

Objective

To design and implement a recommendation system that:

- Suggests movies tailored to individual user preferences.
- Enhances platform retention through accurate and enjoyable recommendations.

Business Problem

Streaming platforms often encounter the following challenges:

- Users are overwhelmed by choice, leading to poor engagement.
- Lack of personalization can increase churn and reduce user satisfaction.
- Recommending popular content may ignore niche preferences, missing opportunities to engage diverse audiences.

An effective recommendation system addresses these challenges by learning from user behavior and content characteristics to provide customized movie suggestions.

Dataset Description:

The MovieLens 100k dataset, curated by the GroupLens research group, contains 100,000 ratings (1–5) from 943 users on 1,682 movies. It includes the following components:

User Information:

User ID

- Age
- Gender
- Occupation
- Zip-code

Movie Information

- Movie ID
- Title
- Genre(s)
- Release Date

Ratings:

- User ID
- Movie ID
- Rating (1–5)
- Timestamp

The MovieLens dataset is clean and well-structured across all files.

- The main ratings (dataset), user info (user), and genre mapping (genre) contain no null values. The movie metadata (item) has minor issues: one missing release date, three missing IMDb URLs, and all values missing in the `video_release_date` column.
- Overall, the data is in good shape for building a recommendation system, with only minimal cleanup needed.

Data Source:

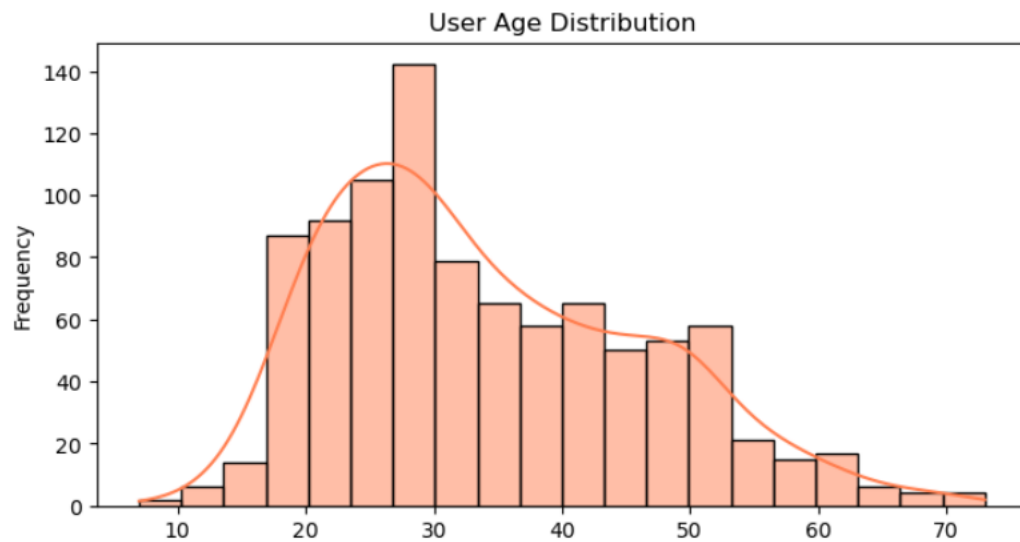
[MovieLens 100k Dataset – Kaggle](#)

Exploratory Data Analysis:

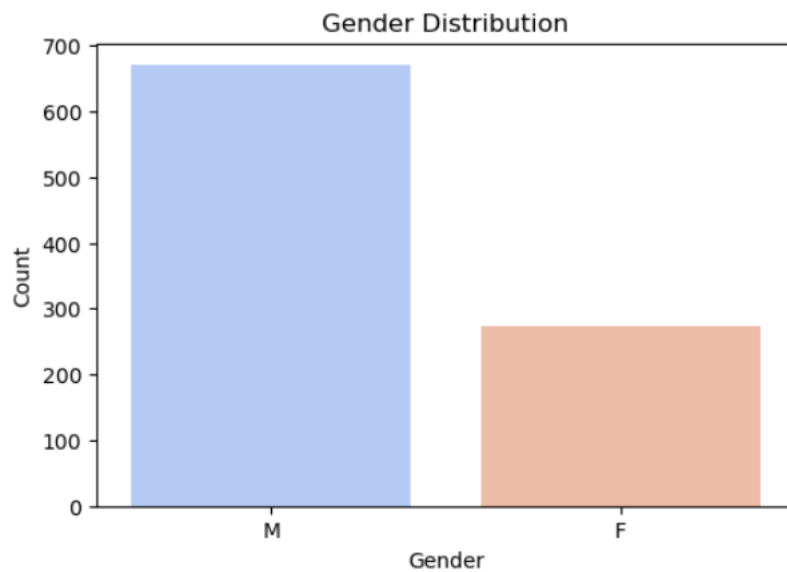
- Our analysis of the movie platform's user base and rating patterns revealed several key insights. The most active demographic on the platform consists of users aged between 25 and 30, indicating a strong engagement from young adults.
- Gender-wise, male users represent the majority of the audience, and among various occupations, students are the most active consumers of movies.
- When analyzing movie ratings, we initially identified *Star Wars (1977)* as the highest-rated film. However, simple average ratings can be misleading—especially for movies with very few ratings, such as *Great Day in Harlem*, which had only one rating but appeared at the top due to its perfect score. To address this issue, we applied the Bayesian Average method, which accounts for both the number of ratings and the overall mean, providing a more balanced view.
- Using the Bayesian formula, which adjusts scores based on our prior knowledge of the average rating (3.08) and average number of ratings (59.45), we determined a more reliable list of top-rated movies.
- The highest rated titles using this method include *Star Wars (1977)*, *Schindler's List (1993)*, and *Shawshank Redemption (1994)*—films that not only received high scores but did so across a large number of ratings, indicating consistent popularity.
- In contrast, movies such as *Leave It to Beaver (1997)* and *McHale's Navy (1997)* were among the lowest rated by Bayesian average, supported by sufficient rating volume to consider these results trustworthy.

- Additionally, a genre-level analysis showed that the **Film-Noir** genre received the highest average ratings, while **Fantasy** had the lowest, reflecting varying audience preferences across movie categories.

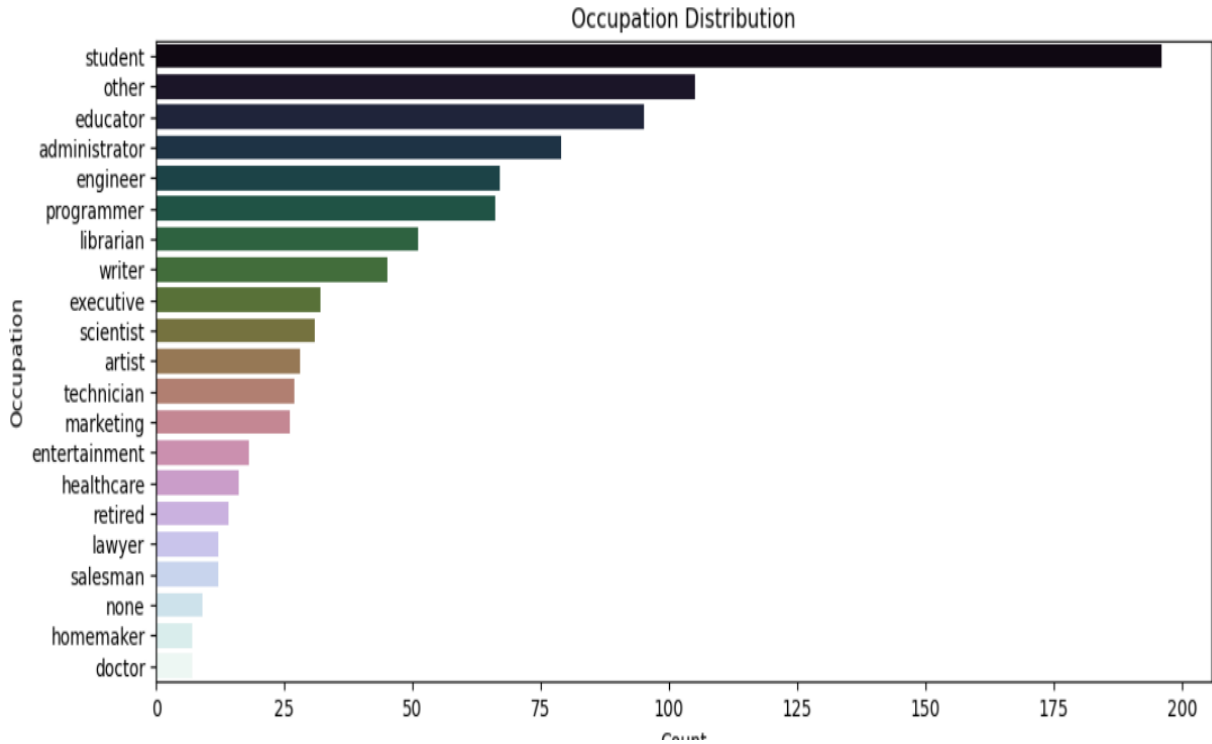
Below are the graphs for insights:



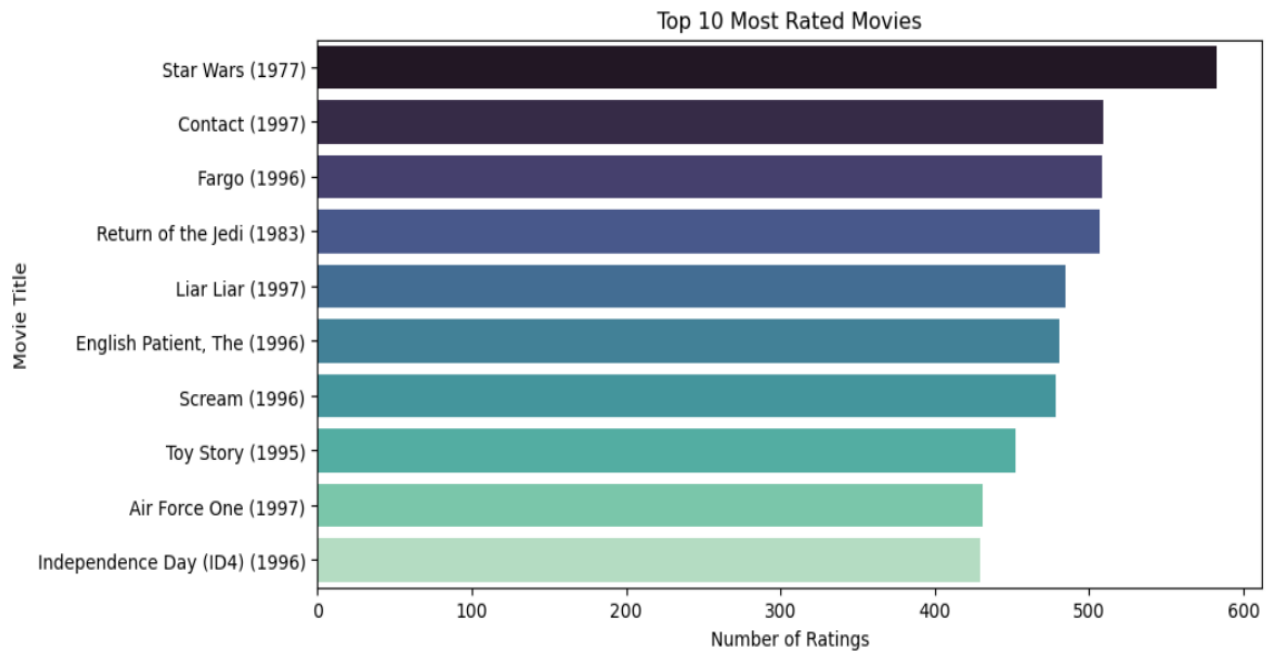
Population with age between 25-30 are the highest user of the movie platform



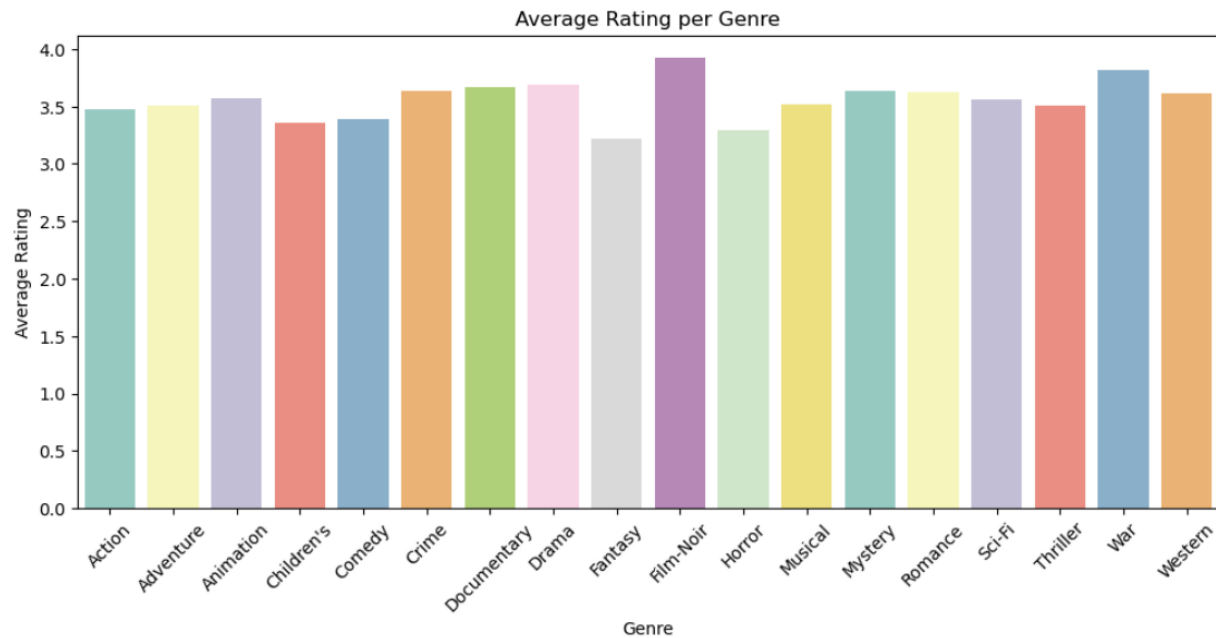
Males are the highest consumers of the movie platform



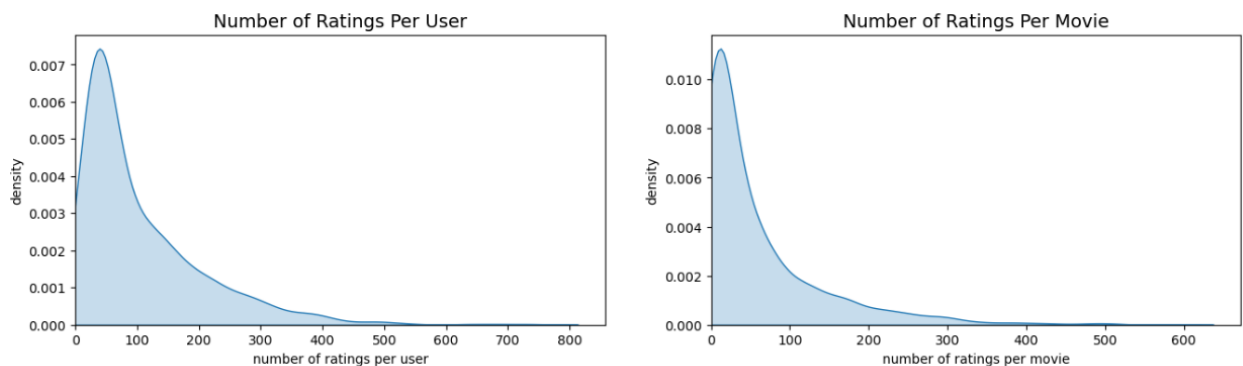
Students are the highest consumer of the movie platform



Above are the top 10 rated movies - Highest is Star Wars (1977)



Ratings are highest for Film-Noir Genre & lowest for Fantasy Genre



Building Recommendation system

Data Preprocessing and Sparsity

We began by converting the user-item-rating matrix into a **sparse matrix** using `scipy.sparse.csr_matrix`, as most users rated only a small fraction of available movies. The resulting matrix had a **sparsity of 6.3%**, indicating that only a small subset of user-movie pairs had known ratings. The most active user rated 737 movies, while the least active rated only 20.

Item-Item Recommendations using k-Nearest Neighbors (k-NN)

In this approach, we use **k-Nearest Neighbors (k-NN)** for **item-item collaborative filtering**. The goal is to recommend movies similar to a given movie based on the **user-item interaction matrix** (e.g., ratings). Instead of predicting ratings, we identify **movies that are most similar** to the movie a user has already liked/watched.

Method Overview:

Item-Item Collaborative Filtering using k-NN:

- Representing each item (movie) as a vector of user ratings.
- Measuring similarity between item vectors (using a distance metric like **cosine similarity**).
- Recommending the top k most similar items (excluding the item itself).

Steps to implement the recommendation system:

- **Start with a user-item matrix**, where rows are users, columns are movies, and values are ratings.
- **Transpose the matrix** so each movie becomes a vector of user ratings.
- **Pick the target movie** for which you want recommendations.
- **Use k-Nearest Neighbors (k-NN)** to find the most similar movies based on rating patterns, using a similarity metric like cosine similarity.
- **Exclude the target movie** from the results (as it's always the closest to itself).
- **Map the similar movie indices** back to movie titles.
- **Display the similar movies** as recommendations based on the one the user watched.

Output for the recommendation system:

Because you watched Toy Story (1995):

Star Wars (1977)

Return of the Jedi (1983)

Independence Day (ID4) (1996)

Rock, The (1996)

Willy Wonka and the Chocolate Factory (1971)

Mission: Impossible (1996)

Fargo (1996)

Raiders of the Lost Ark (1981)

Star Trek: First Contact (1996)

Movie Recommendation using SVD (Singular Value Decomposition)

SVD is a matrix factorization technique used to predict user ratings for unseen items by learning latent factors from known user-item interactions. It helps in building personalized recommendations by identifying patterns in user preferences.

Steps Taken:

- Loaded training and test data for 5 folds.
- Converted data into the required format for the Surprise library.
- Trained the SVD model on each training set.

- Predicted ratings on the corresponding test set.
- Calculated RMSE and MAE for model evaluation.
- Identified movies the user hasn't watched.
- Recommended top 10 movies with the highest predicted ratings.

Output for the recommendation system:

Top 10 Movie Recommendations:

1. Amadeus (1984)
2. Casablanca (1942)
3. One Flew Over the Cuckoo's Nest (1975)
4. Close Shave, A (1995)
5. Wallace & Gromit: The Best of Aardman Animation (1996)
6. Maltese Falcon, The (1941)
7. Paths of Glory (1957)
8. North by Northwest (1959)
9. Manchurian Candidate, The (1962)
10. Raise the Red Lantern (1991)

Movie Recommendation Using Collaborative filtering

The code uses **Item-Based Collaborative Filtering** with **cosine similarity** to recommend movies. It predicts a user's rating for unseen movies based on how similar they are to movies the user has already rated.

Steps Taken:

- Merged user, movie, and ratings data into one dataset.
- Created a **User-Movie rating matrix**.
- Calculated **cosine similarity** between movies.
- Predicted ratings for unseen movies using a **weighted average of similar movies' ratings**.
- Recommended the **top N highest predicted-rated movies** the user hasn't seen.

Output for the recommendation system:

- Star Kid (1997)
- Big Bang Theory, The (1994)
- Other Voices, Other Rooms (1997)
- Men of Means (1998)
- Tokyo Fist (1995)
- Hugo Pool (1997)
- Hush (1998)
- Nightwatch (1997)

- Entertaining Angels: The Dorothy Day Story (1996)
- Further Gesture, A (1996)

Conclusion:

In this report, we explored various collaborative filtering techniques to build movie recommendation systems. We implemented **item-item collaborative filtering using k-Nearest Neighbors (k-NN)**, **matrix factorization with Singular Value Decomposition (SVD)**, and **memory-based collaborative filtering using cosine similarity**. Each method aimed to predict user preferences based on historical rating patterns and recommend movies the user is likely to enjoy.

Through these approaches, we successfully generated personalized movie recommendations. The SVD method offered a more robust prediction by capturing latent factors, while the memory-based methods like cosine similarity and k-NN provided interpretable, similarity-based suggestions. These techniques demonstrate the practical application of recommender systems in enhancing user experience by delivering relevant and tailored content.